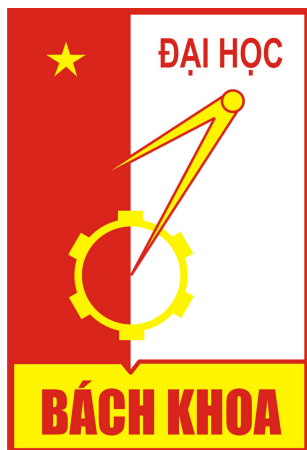


ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG



MÔN HỌC: PROJECT 3

TÊN ĐỀ TÀI:

TruckDrone - DynamicPickup

Giáo viên hướng dẫn: PGS.TS Nguyễn Khánh Phương
Lương Sỹ Linh 20225029

Hà Nội, ngày 15 tháng 1 năm 2026

Mục lục

Lời nói đầu	3
1 Giới thiệu đề tài	4
1.1 Đặt vấn đề	4
1.2 Mục tiêu đề tài	4
1.3 Ý nghĩa đề tài	4
2 Nghiên cứu liên quan	5
2.1 Bài toán định tuyến phối hợp truck-drone	5
2.2 Time window và bài toán định tuyến động	5
2.3 Học/tiến hoá heuristic và Genetic Programming Hyper-Heuristic	6
2.4 Khoảng trống nghiên cứu và định hướng của đề tài	6
3 Phương pháp nghiên cứu	8
3.1 Tổng quan hướng tiếp cận	8
3.2 Mô hình bài toán và các giả định	8
3.2.1 Tập yêu cầu và tham số	8
3.2.2 Đội phương tiện và giả định vận hành	8
3.2.3 Thời gian di chuyển và ràng buộc tour	8
3.3 Tiêu chí và hàm đánh giá	9
3.3.1 Chỉ số đánh giá	9
3.3.2 Hàm mục tiêu	9
3.4 Mô phỏng sự kiện cho môi trường động	9
3.5 Thiết kế heuristic và Genetic Programming	10
3.5.1 Luật điều phối online (Routing & Sequencing)	10
3.5.2 Đặc trưng đầu vào	10
3.5.3 Biểu diễn và huấn luyện Genetic Programming	13
3.6 Quy trình huấn luyện và kiểm thử	15
3.6.1 Thiết lập thí nghiệm	15
3.6.2 Tiêu chí chọn mô hình	15
3.6.3 Baseline so sánh	15
4 Kết quả và đánh giá	16
4.1 Thiết kế thí nghiệm	16
4.1.1 Tập instance và dữ liệu	16
4.1.2 Ràng buộc và hàm mục tiêu	16
4.1.3 Cấu hình Genetic Programming	16
4.1.4 Kịch bản so sánh và ghi nhận kết quả	16
4.1.5 Cách chạy	17



4.2	Kết quả tổng hợp	17
4.3	Phân tích chi tiết và thảo luận	17
4.4	So sánh với baseline và kết luận thực nghiệm	17
5	Hạn chế và hướng phát triển	17
5.1	Hạn chế	17
5.2	Hướng phát triển	17
5.3	Kết luận	17

Lời nói đầu

Trong những năm gần đây, sự phát triển của AI và các hệ thống tự động đã thúc đẩy mạnh mẽ nhu cầu tối ưu hoá vận hành trong logistics. Bên cạnh phương thức vận chuyển truyền thống, mô hình kết hợp xe tải và drone ngày càng được quan tâm nhờ khả năng rút ngắn thời gian phản hồi và tăng tính linh hoạt khi phục vụ khách hàng phân tán. Tuy nhiên, trong thực tế, các yêu cầu thường phát sinh theo thời gian và bị ràng buộc bởi cửa sổ phục vụ (time window), khiến bài toán lập lịch trở nên phức tạp và khó tối ưu bằng các chiến lược tĩnh.

Xuất phát từ bối cảnh đó, mục tiêu của đề tài là nghiên cứu bài toán lập lịch thu gom động cho đội phương tiện kết hợp truck-drone khi các yêu cầu xuất hiện ngẫu nhiên theo thời gian và chịu ràng buộc thời gian phục vụ. Trên cơ sở xây dựng môi trường mô phỏng phản ánh cơ chế phát sinh yêu cầu và quá trình ra quyết định trực tuyến, đề tài tập trung phát triển các luật điều phối ở mức heuristic (chọn phương tiện, chọn điểm phục vụ) và sử dụng Genetic Programming để tự động tiến hoá các luật này. Kết quả được đánh giá theo các tiêu chí tổng quát như thời gian hoàn thành và tỷ lệ yêu cầu được phục vụ. Về phạm vi, đề tài thực nghiệm trên các bộ dữ liệu/instance được thiết kế theo bối cảnh truck-drone có ràng buộc time window, nhằm quan sát mức độ thích ứng của heuristic trong nhiều kịch bản khác nhau. Đóng góp kỳ vọng là cung cấp một hướng tiếp cận tổng quát để tạo ra các luật điều phối có khả năng thích nghi với môi trường động, đồng thời xây dựng nền tảng mô phỏng phục vụ cho việc so sánh và mở rộng về sau. Em xin chân thành cảm ơn TS. Nguyễn Khánh Phương hướng dẫn đã hỗ trợ và góp ý trong quá trình thực hiện đề tài.

Sinh viên thực hiện đề tài

1 Giới thiệu đề tài

1.1 Đặt vấn đề

Sự phát triển của các hệ thống tự động và AI đang thúc đẩy nhu cầu tối ưu hoá vận hành trong logistics, đặc biệt ở các tác vụ “last-mile” vốn nhạy với thời gian và biến động nhu cầu. Mô hình phối hợp *truck-drone* được quan tâm vì có thể kết hợp ưu điểm của xe tải (tải lớn, phạm vi hoạt động rộng) và drone (linh hoạt, phản hồi nhanh), từ đó rút ngắn thời gian phục vụ và cải thiện chất lượng dịch vụ [1], [2], [3]. Tuy nhiên, trong thực tế, các yêu cầu thường phát sinh theo thời gian (động) và đi kèm ràng buộc của sổ thời gian phục vụ (*time window*). Khi đó, bài toán không chỉ là tìm một lộ trình tối ưu ở trạng thái tĩnh, mà còn là ra quyết định trực tuyến: nhận/không nhận yêu cầu, gán phương tiện nào phù hợp, và sắp thứ tự phục vụ thế nào để hạn chế vi phạm thời gian và giảm yêu cầu bị bỏ [4], [5].

1.2 Mục tiêu đề tài

Mục tiêu của đề tài là nghiên cứu bài toán lập lịch thu gom/điều phối động cho đội phương tiện hỗn hợp truck-drone dưới ràng buộc *time window* và các giới hạn vận hành. Trên cơ sở xây dựng môi trường mô phỏng phản ánh tiến trình phát sinh yêu cầu theo thời gian và cơ chế ra quyết định trực tuyến, em hướng tới thiết kế một chiến lược điều phối ở mức heuristic, có khả năng thích ứng với nhiều kịch bản khác nhau.

Cụ thể, em tiếp cận theo hướng học/tiến hoá luật quyết định (ví dụ luật gán phương tiện và luật chọn điểm phục vụ tiếp theo), nhằm đồng thời: (i) giảm thời gian hoàn thành (*makespan*/độ dài lịch trình), (ii) tăng tỷ lệ yêu cầu được phục vụ (giảm số yêu cầu bị loại).

1.3 Ý nghĩa đề tài

Về ý nghĩa thực tiễn, đề tài góp phần làm rõ cách mô hình hoá và đánh giá một hệ thống truck-drone trong bối cảnh nhu cầu động và ràng buộc thời gian, gần với điều kiện vận hành thực. Về ý nghĩa học thuật, đề tài cung cấp một hướng tiếp cận tổng quát dựa trên mô phỏng và tiến hoá heuristic/policy, có thể mở rộng để so sánh với các metaheuristic hoặc các phương pháp học tăng cường trong tương lai [6], [7].

2 Nghiên cứu liên quan

2.1 Bài toán định tuyến phối hợp truck-drone

Các bài toán định tuyến có drone hỗ trợ thường bắt đầu từ các mô hình kinh điển như *Flying Sidekick Traveling Salesman Problem (FSTSP)* và các biến thể VRP-drone, nhấn mạnh đồng bộ giữa truck và drone trong phục vụ khách hàng [1]. Nhiều công trình tổng quan đã hệ thống hoá các hướng mô hình hoá, ràng buộc (tầm bay, tải trọng, đồng bộ chờ) và mục tiêu tối ưu trong bài toán drone-aided routing/truck-drone routing [2], [3].

Gần đây, các biến thể mở rộng cũng được nghiên cứu mạnh, như bài toán nhiều drone song song hoặc bài toán lập lịch drone song song trong bối cảnh VRP, cho thấy mức độ phức tạp tăng nhanh khi bổ sung ràng buộc phối hợp và lịch trình [8]. Ngoài ra, các hướng nghiên cứu về đồng bộ động và độ tin cậy khi có bất định thời gian di chuyển cũng được quan tâm [9].

2.2 Time window và bài toán định tuyến động

Đối với bài toán định tuyến xe có cửa sổ thời gian (VRPTW), mỗi khách hàng i chỉ được phép bắt đầu phục vụ trong khoảng $[e_i, l_i]$, do đó nghiệm không chỉ cần tối ưu hoá chi phí (quãng đường/thời gian) mà còn phải thoả ràng buộc thời gian chặt chẽ. Đây là một lớp bài toán kinh điển trong tối ưu tổ hợp và đã được nghiên cứu rộng rãi với nhiều hướng tiếp cận, từ mô hình hoá chính xác (exact) đến heuristic/metaheuristic, nhằm cân bằng giữa chất lượng nghiệm và thời gian tính toán [5]. Điểm khó cốt lõi của VRPTW nằm ở tính “liên kết” của ràng buộc: chỉ một thay đổi nhỏ trong thứ tự ghé thăm cũng có thể lan truyền và gây vi phạm time window ở nhiều điểm sau đó, khiến không gian nghiệm bị cắt mạnh và dễ rơi vào các cực trị cục bộ.

Khi chuyển sang bối cảnh động (DVRP/DVRPTW), bài toán trở thành một quy trình ra quyết định theo thời gian thực: khách hàng/yêu cầu xuất hiện dần theo thời gian, thông tin tương lai chưa biết, và hệ thống phải liên tục cập nhật lịch trình để tiếp nhận hoặc từ chối yêu cầu mới. Thách thức chính không chỉ là “tối ưu hiện tại” mà là cân bằng giữa (i) cải thiện lịch trình hiện thời và (ii) duy trì mức linh hoạt để tiếp nhận các yêu cầu sắp tới, trong khi vẫn đảm bảo tính khả thi theo time window [4]. Do đó, mô phỏng và đánh giá online (theo dòng thời gian) thường được xem là thành phần quan trọng để phản ánh đúng hành vi của thuật toán trong môi trường động, thay vì chỉ đánh giá trên một bài toán tĩnh đã biết đầy đủ.

Trong bối cảnh có drone, time window làm gia tăng đáng kể độ khó do xuất hiện thêm các ràng buộc vận hành (tầm bay/ năng lượng, tải trọng, giới hạn vùng phục vụ, đồng bộ lịch trình). Một nhánh nghiên cứu gần đây xem xét các biến thể VRP-drone-time-window (VRP-DTW), trong đó mục tiêu là phối hợp truck và drone dưới ràng buộc thời gian để giảm thời gian hoàn thành hoặc chi phí, và đề xuất cả phương pháp exact lẫn heuristic tùy kích thước bài toán [10]. Một số công trình khác đặt trọng tâm vào yếu tố động (dynamic demand) kết hợp time window trong ngữ cảnh VRP có drone, nhấn mạnh rằng quyết định gán (truck hay drone) phải được thực hiện nhanh và có khả năng thích nghi khi yêu cầu đến theo thời gian [11]. Nhìn chung, xu hướng chung là: khi bài toán càng gần thực tế (động, time window,

drone constraints), các chiến lược online dựa trên heuristic/policy trở nên phù hợp hơn về mặt vận hành do yêu cầu phản hồi nhanh và liên tục.

Với bài toán có drone và time window, một số nghiên cứu gần đây xem xét VRP-drone-time-window (VRP-DTW) và đề xuất cả thuật toán exact lẫn heuristic để xử lý đồng bộ truck/drone dưới ràng buộc thời gian [10]. Các công trình khác cũng đặt trọng tâm vào nhu cầu động kết hợp time window trong ngữ cảnh VRP có drone [11].

2.3 Học/tiến hoá heuristic và Genetic Programming Hyper-Heuristic

Trong các môi trường động, heuristic dạng “luật quyết định” (dispatching/routing rule) thường được ưu tiên vì hai lý do: (i) tốc độ ra quyết định nhanh, có thể áp dụng ở mỗi sự kiện (yêu cầu mới xuất hiện, phương tiện rảnh, v.v.), và (ii) dễ triển khai trong mô phỏng/triển khai thực tế do chỉ cần tính một hàm điểm và chọn phương án tốt nhất. Tuy nhiên, nhược điểm của heuristic thiết kế thủ công là phụ thuộc mạnh vào trực giác người thiết kế và có thể không khái quát tốt khi thay đổi phân bố dữ liệu, mức độ động, hoặc mức độ chặt của time window.

Genetic Programming (GP) cung cấp một hướng tiếp cận tự động hoá thiết kế heuristic: thay vì “tự nghĩ” công thức chấm điểm, ta biểu diễn luật quyết định như một biểu thức/cây chương trình kết hợp các đặc trưng trạng thái (khoảng cách, độ gấp theo time window, tải còn lại, mức khả thi của drone, v.v.) và các toán tử cơ bản, rồi dùng tiến hoá để tìm biểu thức có hiệu quả tốt nhất theo một hàm fitness. Các nghiên cứu về tiến hoá dispatching rules cho bài toán định tuyến động cho thấy GP có thể tìm ra các luật có cấu trúc không hiển nhiên nhưng hiệu quả, đồng thời giảm công sức tinh chỉnh thủ công [6].

Trong nhánh *Genetic Programming Hyper-Heuristic* (GPHH), GP không trực tiếp tìm một nghiệm cho từng instance, mà tìm *heuristic tổng quát* có khả năng áp dụng lặp lại trên nhiều instance/kịch bản. Điều này đặc biệt phù hợp với DVRPTW, nơi thuật toán phải xử lý chuỗi sự kiện và điều chỉnh lịch trình liên tục. Một số nghiên cứu đã áp dụng GPHH để tiến hoá heuristic phục vụ các quyết định online như: chèn yêu cầu mới, chấp nhận/từ chối yêu cầu, và lựa chọn thứ tự phục vụ nhằm giảm vi phạm time window và tăng chất lượng lịch trình, cho thấy tiềm năng vượt trội so với các heuristic thủ công trong một số thiết lập [7]. Bên cạnh đó, các hướng GPHH cho các bài toán định tuyến/arc routing dưới bất định cũng củng cố lập luận rằng “tiến hoá chính sách” là cách tiếp cận phù hợp khi trạng thái thay đổi liên tục và bài toán chứa nhiều ràng buộc đồng thời [12].

Một điểm thực tiễn quan trọng là: để GPHH hiệu quả, cần một môi trường đánh giá (fitness) phản ánh đúng cơ chế vận hành (online, theo thời gian, có sự kiện), và một tập đặc trưng đủ giàu để luật có thể phân biệt các tình huống (gấp theo time window, mức độ khả thi drone, rủi ro gây trễ cho các khách sau). Do vậy, việc xây dựng mô phỏng sự kiện và thiết kế tập đặc trưng/fitness đóng vai trò then chốt để biến GPHH thành một công cụ tạo heuristic có khả năng khái quát.

2.4 Khoảng trống nghiên cứu và định hướng của đề tài

Tổng hợp các hướng nghiên cứu cho thấy còn tồn tại một số khoảng trống và động lực triển khai đề tài như sau:

- **Khoảng trống 1 — Tích hợp đồng thời nhiều yếu tố gần thực tế:** Truck-drone routing đã phong phú về mô hình và ràng buộc, VRPTW/DVRPTW cũng có nền tảng mạnh; tuy nhiên khi kết hợp đồng thời (i) *yêu cầu động*, (ii) *time window chặt*, và (iii) *phối hợp truck-drone* thì độ phức tạp tăng đáng kể và yêu cầu chiến lược ra quyết định online rõ ràng.
- **Khoảng trống 2 — Nhu cầu về heuristic online có khả năng khái quát:** Trong môi trường động, thuật toán cần phản hồi nhanh ở mỗi sự kiện. Các heuristic thủ công thường khó bao phủ hết nhiều kịch bản (mức động khác nhau, phân bố time window khác nhau, tỷ lệ truck-only khác nhau), do đó cần một cơ chế tạo luật có tính thích nghi.
- **Khoảng trống 3 — Đánh giá phải bám theo mô phỏng theo thời gian:** Nhiều kết quả tốt trên mô hình tĩnh có thể suy giảm khi chuyển sang online. Vì vậy, việc xây dựng mô phỏng sự kiện để đánh giá trực tiếp theo tiến trình thời gian là cần thiết để phản ánh đúng hiệu quả vận hành.

Từ các nhận định trên, đề tài lựa chọn hướng tiếp cận **xây dựng mô phỏng sự kiện cho môi trường động** và **tiến hoá luật điều phối bằng GP** dưới dạng cặp luật *routing/sequencing*. Mục tiêu là tối ưu đồng thời *thời gian hoàn thành (makespan)* và *tỷ lệ phục vụ* trong bối cảnh yêu cầu phát sinh theo thời gian và chịu nhiều ràng buộc vận hành.

3 Phương pháp nghiên cứu

3.1 Tổng quan hướng tiếp cận

Đề tài tiếp cận theo hướng **mô phỏng sự kiện** (event-driven simulation) để mô hình hoá môi trường yêu cầu động, kết hợp **heuristic điều phối online** để ra quyết định theo thời gian thực, và sử dụng **Genetic Programming (GP)** nhằm tiến hoá cặp luật quyết định gồm *routing* và *sequencing*. Mỗi cá thể (một cặp luật) được đánh giá bằng mô phỏng trên tập instance, từ đó chọn ra heuristic có hiệu năng tốt và ổn định.

3.2 Mô hình bài toán và các giả định

3.2.1 Tập yêu cầu và tham số

Xét một depot ký hiệu 0 và tập yêu cầu/khách hàng $V = \{1, \dots, n\}$. Mỗi yêu cầu $i \in V$ có:

- Toạ độ (x_i, y_i) và nhu cầu (khối lượng) $q_i > 0$;
- Cửa sổ thời gian phục vụ $[e_i, l_i]$ và thời gian phục vụ $s_i \approx 0$;
- Thời điểm xuất hiện (release time) r_i trong môi trường động.

Tập khách chỉ phục vụ bằng truck ký hiệu $V_T \subseteq V$ (*truck-only*); các khách còn lại $V \setminus V_T$ có thể được phục vụ bởi drone nếu thỏa điều kiện tầm bay.

3.2.2 Đối phương tiện và giả định vận hành

Đối phương tiện gồm m_T truck và m_D drone, với:

$$(v_T, v_D) \text{ là tốc độ, } (M_T, M_D) \text{ là tải tối đa.}$$

Drone có bán kính/tầm bay tối đa R tính từ depot; một điểm i là hợp lệ cho drone nếu:

$$\text{dist}(0, i) \leq R.$$

Giả định mọi hành trình bắt đầu và kết thúc tại depot; không xét cơ chế rendezvous (launch/recover) giữa truck và drone.

3.2.3 Thời gian di chuyển và ràng buộc tour

Khoảng cách giữa hai điểm $i, j \in \{0\} \cup V$ được ký hiệu $\text{dist}(i, j)$. Thời gian di chuyển của từng loại phương tiện:

$$t_{ij}^T = \frac{\text{dist}(i, j)}{v_T}, \quad t_{ij}^D = \frac{\text{dist}(i, j)}{v_D}.$$

Một yêu cầu có thể bị *bỏ* (dropped) nếu không thể gán hợp lệ trong quá trình vận hành. Đặt giới hạn L_w cho một tour: với mỗi phương tiện, thời gian hoàn thành một tour (từ lúc rời depot đến lúc quay lại depot) không vượt quá L_w . Trong thực nghiệm, sử dụng $L_w = 3600$ s.

3.3 Tiêu chí và hàm đánh giá

3.3.1 Chỉ số đánh giá

Gọi $C_{\max}$ là makespan (thời điểm kết thúc muộn nhất của mọi phương tiện). Biến $y_i \in \{0, 1\}$ cho biết yêu cầu i được phục vụ ($y_i = 1$) hay bị bỏ ($y_i = 0$). Tỷ lệ phục vụ:

$$\text{ServiceRate} = \frac{\sum_{i \in V} y_i}{|V|}.$$

3.3.2 Hàm mục tiêu

Hàm mục tiêu dùng trong mô phỏng:

$$\min \lambda_w \frac{C_{\max}}{T_{\text{ref}}} + \gamma(1 - \text{ServiceRate}),$$

trong đó T_{ref} là makespan của nghiệm baseline. Các hệ số λ_w và γ được đọc từ `config.yaml` (thông qua `objective.lambda_w` và `drop_penalty`) nhằm cân bằng giữa rút ngắn lịch trình và giảm số yêu cầu bị bỏ.

3.4 Mô phỏng sự kiện cho môi trường động

Môi trường được mô hình hoá theo mô phỏng sự kiện rời rạc, trong đó đồng hồ hệ thống nhảy tới (i) thời điểm xuất hiện yêu cầu tiếp theo hoặc (ii) thời điểm một phương tiện trở nên rảnh. Tại mỗi mốc thời gian, hệ thống thực hiện các cơ chế sau:

- **Xử lý yêu cầu mới** ($t \geq r_i$): với mỗi yêu cầu mới, hệ thống ước lượng các phương án *chèn khả thi* (insertion) vào hàng đợi của từng phương tiện, sau đó gọi luật *routing* để chọn *phương tiện* (và *vị trí chèn*) tốt nhất. Nếu không tồn tại phương án chèn khả thi tại thời điểm hiện tại, yêu cầu được đưa vào tập *pending* để chờ; yêu cầu chỉ bị loại khi đã quá hạn trong pending.
- **Sự kiện phương tiện rảnh**: khi một phương tiện trở nên rảnh, trước khi phục vụ hàng đợi hiện tại, phương tiện thử “kéo” một số yêu cầu từ *pending* vào hàng đợi nếu tồn tại vị trí chèn khả thi. Sau đó, luật *sequencing* được gọi để chọn khách kế tiếp trong hàng đợi để phục vụ.
- **Kiểm tra khả thi**: một yêu cầu chỉ được chấp nhận (chèn hoặc phục vụ) nếu đồng thời thoả các điều kiện:
 1. **Tải trọng**: tải còn lại không vượt M_T (truck) hoặc M_D (drone);
 2. **Dronable**: drone chỉ nhận các yêu cầu có `drone_ok=1`;
 3. **Tầm bay**: nếu cấu hình bắt ràng buộc `radius` cho drone, yêu cầu phải thoả $\text{dist}(0, i) \leq R$;
 4. **Time window**: nếu đến sớm thì được phép chờ đến e_i , nhưng thời điểm bắt đầu phục vụ không vượt l_i ;
 5. **Giới hạn tour** L_w : tại thời điểm phục vụ, thời gian hoàn thành tour (kể từ khi rời depot cho tới khi quay lại depot) không vượt L_w .
- **Kết thúc tour và loại bỏ**: khi phương tiện quay về depot, hệ thống đặt lại tải và cập nhật thời điểm sẵn sàng tiếp theo. Các yêu cầu trong hàng đợi hoặc pending sẽ bị loại nếu đã quá hạn; các yêu cầu còn lại tới cuối mô phỏng cũng được đánh dấu là không phục vụ.

Hệ thống ghi nhận $C_{\max}$ (makespan), số yêu cầu được phục vụ/bị loại và nguyên nhân loại để tính giá trị hàm mục tiêu. Thời gian phục vụ tại mỗi điểm được giả định xấp xỉ bằng 0.

3.5 Thiết kế heuristic và Genetic Programming

3.5.1 Luật điều phối online (Routing & Sequencing)

Hai luật *routing* (R) và *sequencing* (S) đều hoạt động theo cơ chế *chấm điểm-chọn tốt nhất*: với mỗi phương án khả thi, luật trả về một điểm số; hệ thống chọn phương án có điểm cao nhất.

- **Routing (R):** khi một yêu cầu mới i xuất hiện, hệ thống xét từng phương tiện khả thi k và các vị trí chèn khả thi trong hàng đợi của k . Với mỗi k , lấy phương án chèn tốt nhất (theo tiêu chí khả thi và/hoặc chi phí chèn) để tạo một ứng viên (k, pos^*) . Luật R tính điểm trên vector đặc trưng của ứng viên và chọn phương tiện có điểm cao nhất. Nếu không tồn tại phương án chèn khả thi, yêu cầu được đưa vào tập *pending* (chờ xử lý); chỉ bị loại khi quá hạn.
- **Sequencing (S):** khi phương tiện k rảnh và hàng đợi không rỗng, luật S tính điểm cho từng khách j trong hàng đợi dựa trên đặc trưng hiện tại, sau đó chọn khách có điểm cao nhất để phục vụ tiếp.

3.5.2 Đặc trưng đầu vào

Mỗi luật nhận một vector đặc trưng $\mathbf{x} = (x_0, \dots, x_{12})$. Các đặc trưng được chuẩn hoá bằng các hằng số $D_{\max}$ (khoảng cách lớn nhất), H (mốc chuẩn hoá thời gian) và $Q_{\max}$ (nhu cầu lớn nhất). Ký hiệu $\text{dist}(v, i)$ là khoảng cách từ vị trí hiện tại của phương tiện v tới yêu cầu i , cap là tải tối đa của phương tiện và load là tải hiện tại.

- x_0 **dist_norm**: $\frac{\text{dist}(v, i)}{D_{\max}}$
- x_1 **demand_norm**: $\frac{q_i}{Q_{\max}}$
- x_2 **rem_capacity_norm**: $\frac{\text{cap} - \text{load}}{q_i}$
- x_3 **demand_over_rc**: $\frac{\text{cap} - \text{load}}{\max(\text{cap} - \text{load}, \varepsilon)}$
- x_4 **nearest_next_norm**: khoảng cách từ i tới khách đang chờ gần nhất (trong queue/pending theo cấu hình) được chuẩn hoá bởi $D_{\max}$
- x_5 **time_to_ready**: $\frac{\max(0, e_i - t)}{H}$
- x_6 **time_to_due**: $\frac{\max(0, l_i - t)}{H}$
- x_7 **waiting_time**: $\frac{\max(0, t - r_i)}{H}$
- x_8 **drone_ok**: 1 nếu yêu cầu cho phép drone, 0 nếu truck-only
- x_9 **is_drone**: 1 nếu phương tiện là drone, 0 nếu truck
- x_{10} **ins_delta_end**: $\frac{\Delta C_{\text{end}}}{H}$, với ΔC_{end} là mức tăng thời điểm kết thúc tour theo phương án chèn tốt nhất
- x_{11} **ins_slack_min**: $\frac{\text{min-slack}}{H}$, với min-slack là slack nhỏ nhất dọc theo tour dự kiến (theo phương án chèn tốt nhất)
- x_{12} **ins_feasible_frac**: tỷ lệ vị trí chèn khả thi trong hàng đợi (số vị trí chèn khả thi chia cho tổng số vị trí chèn)

Algorithm 1: Vòng lặp mô phỏng sự kiện rời rạc (event loop)

Input: Danh sách yêu cầu đã sắp theo t_{arrive} ; đội phương tiện $\{V_k\}$; depot; L_w ; policy (R, S) .

Output: Stats: C_{max} , served/total/dropped, **drop_breakdown**, timeline.

Khởi tạo $t \leftarrow 0$, $\text{next_idx} \leftarrow 0$, $\text{pending} \leftarrow \emptyset$, $\text{served} \leftarrow \emptyset$, $\text{dropped} \leftarrow \emptyset$

Khởi tạo mọi V_k : $V_k.\text{pos} = \text{depot}$, $V_k.\text{load} = 0$, $V_k.\text{free_at} = 0$, $V_k.\text{queue} = \emptyset$

while not AllDone($(\text{next_idx}, \text{pending}, \{V_k.\text{queue}\})$) **do**

$\text{next_arrival} \leftarrow (\text{next_idx} < N) ? t_{\text{arrive}}[\text{next_idx}] : \infty$

$\text{next_free} \leftarrow \min\{V_k.\text{free_at} : V_k.\text{queue} \neq \emptyset \text{ or } V_k.\text{free_at} > t\}$ (nếu rỗng thì ∞)

$t \leftarrow \min(\text{next_arrival}, \text{next_free})$

if $t = \infty$ **then**

break

 // (0) Loại bỏ yêu cầu quá hạn

$\text{pending} \leftarrow \text{ExpireList}((\text{pending}, t, \text{pending_expired}))$

foreach V_k **do**

$V_k.\text{queue} \leftarrow \text{ExpireList}((V_k.\text{queue}, t, \text{expired_waiting_in_queue}, k))$

 // (1) Nhận các yêu cầu mới đến

while $\text{next_idx} < N$ **and** $t_{\text{arrive}}[\text{next_idx}] \leq t$ **do**

foreach V_k **do**

$\text{ins}[k] \leftarrow \text{BestInsertion}((i = \text{next_idx}, V_k, t, L_w))$

$(\text{veh}, \text{pos}) \leftarrow R(\text{features}, \text{ins})$

if AssignRequest($(i = \text{next_idx}, t, \text{veh}, \text{pos})$) **=false** **then**

 AssignRequest($(i = \text{next_idx}, t, \text{None}, \text{None})$)

$\text{next_idx} \leftarrow \text{next_idx} + 1$

 // (2) Xe rảnh: kéo từ pending và phục vụ

foreach V_k **do**

if $V_k.\text{free_at} \leq t$ **then**

 PullFromPending($(k, t, \text{top_k} = 5)$)

 ServeVehicle((k, t, S, L_w))

 // (3) Kết thúc: đánh dấu các yêu cầu còn lại

for $i \leftarrow 0$ **to** $N - 1$ **do**

if $i \notin \text{served}$ **and** $i \notin \text{dropped}$ **then**

 drop($i, \text{unserved_end_of_sim}$)

 Tính $C_{\text{max}} \leftarrow \max_k V_k.\text{free_at}$ và tổng hợp thống kê

Algorithm 2: Thủ tục phục vụ hàng đợi của một phương tiện (sequencing & service)

Input: Phương tiện V_k tại thời điểm t ; luật sequencing S ; giới hạn L_w .

Output: Cập nhật V_k (pos, load, free_at, queue) và các log served/drop/timeline.

if $V_k.free_at > t$ **or** $V_k.queue = \emptyset$ **then**
 $\perp$ **return**

$t_v \leftarrow \max(t, V_k.free_at)$

$pos_v \leftarrow V_k.pos$

$load \leftarrow V_k.load$

while $V_k.queue \neq \emptyset$ **do**
 $V_k.queue \leftarrow \text{ExpireList}((V_k.queue, t_v, \text{expired_waiting_in_queue}, k))$
 if $V_k.queue = \emptyset$ **then**
 $\perp$ **break**
 $nxt \leftarrow S(\text{features}, V_k)$
 if $nxt \notin V_k.queue$ **then**
 $\perp$ **continue**
 if $t_{arrive}[nxt] > t_v$ **then**
 $\perp$ **break**
 if $\text{IsFeasible}((k, nxt, load, t_v, pos_v)) = \text{false}$ **then**
 $\text{drop}(nxt, \text{infeasible_at_service}, k)$
 $V_k.queue \leftarrow V_k.queue \setminus \{nxt\}$
 $\perp$ **continue**
 $start \leftarrow \max(t_v + \text{travel}(pos_v, nxt), e_{nxt})$
 if $start > l_{nxt}$ **then**
 $\text{drop}(nxt, \text{cannot_reach_before_due}, k)$
 $V_k.queue \leftarrow V_k.queue \setminus \{nxt\}$
 $\perp$ **continue**
 if $\text{ViolatesLw}((k, nxt, t_v, pos_v, L_w))$ **then**
 $\text{drop}(nxt, \text{violates_Lw}, k)$
 $V_k.queue \leftarrow V_k.queue \setminus \{nxt\}$
 $\perp$ **continue**
 // Phục vụ
 $t_v \leftarrow start$
 $pos_v \leftarrow (x_{nxt}, y_{nxt})$
 $load \leftarrow load + q_{nxt}$
 ghi timeline (k, nxt, t_v)
 $served \leftarrow served \cup \{nxt\}$
 $V_k.queue \leftarrow V_k.queue \setminus \{nxt\}$
 // Kết thúc tour: quay về depot và đặt lại tải
 $V_k.free_at \leftarrow t_v + \text{travel}(pos_v, \text{depot})$
 $V_k.pos \leftarrow \text{depot}; V_k.load \leftarrow 0$
 $V_k.queue \leftarrow \text{ExpireList}((V_k.queue, V_k.free_at, \text{expired_in_queue_after_tour}, k))$

3.5.3 Biểu diễn và huấn luyện Genetic Programming

- **Biểu diễn cá thể:** mỗi cá thể là một cặp cây GP (R, S) . Tập toán tử bao gồm $+$, $-$, $\times$, chia bảo vệ (*protected division*), cùng các hàm $\max$, $\min$, abs , neg và tanh . Các terminal là các biến đầu vào tương ứng với các đặc trưng $x_0, \dots, x_{12}$.
- **Đánh giá (fitness):** mỗi cá thể được chạy trong mô phỏng sự kiện để thu được makespan và số yêu cầu được phục vụ *served* trên tổng *total*. Hàm đánh giá:

$$F = \lambda_w \frac{\text{makespan}}{T_{\text{ref}}} + \text{drop_penalty} \left(1 - \frac{\text{served}}{\text{total}} \right),$$

trong đó T_{ref} là makespan của baseline (Greedy EDD) trên cùng instance. Hai hệ số λ_w và drop_penalty được đọc từ cấu hình `objective` trong `config.yaml`.

- **Thuật toán tiến hoá:** khởi tạo quần thể kích thước `gp.pop_size`, trong đó trộn các hạt giống `gp.seed_pairs` theo tỷ lệ `gp.seed_ratio`. Chọn lọc bằng giải đấu kích thước `gp.tournament_k`. Lai ghép một điểm (one-point crossover) với xác suất `gp.rc` và đột biến uniform (uniform mutation) với xác suất `gp.rm`. Áp dụng ràng buộc kích thước cây (giới hạn độ dài tối đa 50 node) và chiến lược elitism để giữ cá thể tốt nhất mỗi thế hệ. Thuật toán lặp `gp.generations` thế hệ và sử dụng kích thước quần thể trung gian `gp.intermed_size` theo cấu hình.

Algorithm 3: Huấn luyện GP hyper-heuristic cho cặp luật (R, S)

Input: Cấu hình `gp.*`, `objective.*`, tập terminal $\{x_0, \dots, x_{12}\}$ và toán tử.

Output: Cặp cây GP tốt nhất (R^*, S^*) .

Biểu diễn cá thể: mỗi cá thể là cặp cây (R, S) ; toán tử $\{+, -, \times, \div$ bảo vệ, $\max, \min, \text{abs}, \text{neg}, \text{tanh}\}$;
terminal là các đặc trưng $x_0..x_{12}$; giới hạn kích thước cây tối đa 50 node

```
// 1) Khởi tạo toolbox và quần thể
tb ← BuildToolbox(operators, terminals, gp.max_depth)
pop ← InitPopulation(gp.pop_size, gp.seed_pairs, gp.seed_ratio)
fit ← EvaluatePopulation(pop) // đánh giá bằng mô phỏng sự kiện và công thức fitness

for g ← 1 to gp.generations do
    // 2) Elitism: lưu cá thể tốt nhất của quần thể hiện tại
    idx* ← arg mini fit[i]
    elite ← pop[idx*]

    // 3) Chọn tập cha mẹ (intermediate pool)
    parents ← TournamentSelect(pop, fit, gp.intermed_size, gp.tournament_k)

    // 4) Sinh offspring tối đủ kích thước quần thể
    offspring ← ∅
    while |offspring| < gp.pop_size do
        Chọn ngẫu nhiên a, b ∈ parents
        (aR, aS) ← a, (bR, bS) ← b

        if rand < gp.rc then
            (aR, bR) ← OnePointCrossover((aR, bR))
            (aS, bS) ← OnePointCrossover((aS, bS))

        if rand < gp.rm then
            aR ← UniformMutation((aR))

        if rand < gp.rm then
            aS ← UniformMutation((aS))

        (aR, aS) ← StaticLimit((aR, aS)) // giới hạn kích thước cây (static limit)
        Thêm (aR, aS) vào offspring

    // 5) Đánh giá offspring và chèn elite
    pop ← offspring
    fit ← EvaluatePopulation(pop)
    idxworst ← arg maxi fit[i]
    pop[idxworst] ← elite; fit[idxworst] ← fitness của elite
    Ghi nhận Fbest(g) ← mini fit[i]

idx* ← arg mini fit[i]; (R*, S*) ← pop[idx*]
return (R*, S*)
```

3.6 Quy trình huấn luyện và kiểm thử

3.6.1 Thiết lập thí nghiệm

Quá trình huấn luyện GP được thực hiện trên một instance được chỉ định trong `config.yaml` (trường `instance`). Các siêu tham số của GP (kích thước quần thể, số thế hệ, xác suất lai ghép/đột biến, hạt giống, v.v.) được đọc từ nhóm cấu hình `gp.*`. Giá trị tham chiếu T_{ref} được tính bằng cách chạy baseline Greedy EDD trên cùng instance trước khi huấn luyện, và được sử dụng để chuẩn hoá thành phần makespan trong hàm fitness.

Để đánh giá khả năng khái quát, mô hình sau huấn luyện có thể được chạy lại trên: (i) nhiều instance khác nhau (thay đổi `instance`), hoặc (ii) nhiều hạt giống ngẫu nhiên khác nhau (thay đổi `gp.seed`), sau đó tổng hợp và so sánh các chỉ số đánh giá.

3.6.2 Tiêu chí chọn mô hình

Trong mỗi thế hệ, thuật toán lưu lại cá thể tốt nhất theo giá trị fitness F (bài toán tối thiểu hoá). Sau khi kết thúc `gp.generations` thế hệ, cá thể có F nhỏ nhất trong toàn bộ quá trình tiến hoá được chọn làm mô hình cuối cùng.

Hiện tại hệ thống chưa tách tập *validation* nội bộ; do đó, để kiểm tra độ ổn định và hạn chế overfitting theo một kịch bản, cần thực hiện đánh giá lại trên các seed/instance khác và báo cáo thống kê (trung bình, độ lệch chuẩn) của các chỉ số như makespan và tỷ lệ phục vụ.

3.6.3 Baseline so sánh

Baseline mặc định là Greedy EDD (Earliest Due Date), ưu tiên phục vụ các yêu cầu có hạn chót sớm hơn trong số các phương án khả thi. Việc so sánh được thực hiện dựa trên:

- **Makespan** C_{max} ;
- **Tỷ lệ phục vụ** $\text{ServiceRate} = \frac{\text{served}}{\text{total}}$;
- **Thống kê yêu cầu bị loại (drop)**: tổng số lượng và phân rã theo lý do drop.

4 Kết quả và đánh giá

4.1 Thiết kế thí nghiệm

4.1.1 Tập instance và dữ liệu

Các thí nghiệm được thực hiện trên **tập instance** lưu trong thư mục `data/raw/`, mỗi instance tương ứng một thư mục con (ví dụ: `50.40.1/`, `50.40.2/`, ...) chứa tệp `requests.csv`. Thông tin tĩnh (depot, đội phương tiện, cấu hình ban đầu) được đọc từ `inputs_static/<instance>.json` tương ứng.

Hệ thống hỗ trợ hai chế độ chạy:

- **Chạy batch (mặc định cho đánh giá):** duyệt toàn bộ (hoặc một tập con) các instance trong `data/raw/` và tổng hợp kết quả.
- **Chạy đơn instance (phục vụ phân tích/debug):** chỉ định instance trong `config.yaml` (ví dụ `50.40.1`) để chạy và quan sát chi tiết `timeline` và `drop_breakdown`.

4.1.2 Ràng buộc và hàm mục tiêu

Các ràng buộc chính trong mô phỏng:

- Giới hạn tour: $L_w = 3600$ s (từ `constraints.Lw`);
- Thời gian phục vụ tại mỗi điểm được giả định xấp xỉ bằng 0.

Hàm fitness dùng để đánh giá (và huấn luyện GP):

$$F = \lambda_w \cdot \frac{\text{makespan}}{T_{\text{ref}}} + \text{drop_penalty} \cdot \left(1 - \frac{\text{served}}{\text{total}}\right),$$

trong đó $\lambda_w = 0.05$, `drop_penalty` = 30.0, và T_{ref} là makespan của baseline Greedy EDD tính trên cùng instance. Chế độ mục tiêu sử dụng `objective.mode = weighted` với `weights.makespan = weights.unserved = 0.5`.

4.1.3 Cấu hình Genetic Programming

Các tham số GP được lấy từ `config.yaml`:

- **Tiến hoá:** `pop_size` = 200, `intermed_size` = 30, `generations` = 30.
- **Cấu hình cây:** `max_depth` = 5, `rc` = 0.6, `rm` = 0.4, `tournament_k` = 2.
- **Hạt giống khởi tạo:** `seed` = 42, `seed_ratio` = 0.15, danh sách `seed_pairs` gồm 12 cặp biểu thức khởi tạo sẵn.

4.1.4 Kịch bản so sánh và ghi nhận kết quả

GP-heuristic được so sánh với baseline Greedy EDD trên cùng tập instance và cùng cấu hình mô phỏng.

Các chỉ số ghi nhận gồm:

- Makespan và tỷ lệ phục vụ `served/total`;
- Số lượng dropped và phân rã theo lý do (`drop_breakdown`);
- Timeline phục vụ (phục vụ phân tích hành vi và trực quan hoá).

Kết quả được lưu tại `results/_batch/` (khi chạy batch) hoặc `results/$INSTANCE/` (khi chạy đơn instance). Nếu tồn tại `benchmark.json`, hệ thống xuất kèm các thống kê so sánh với benchmark.

4.1.5 Cách chạy

- Huấn luyện GP (theo instance cấu hình): `python -m src.cli train-gp -config config.yaml`
- Đánh giá chạy đơn instance: `python -m src.cli eval -config config.yaml`
- Đánh giá chạy batch trên tập `data/raw/`: `python -m src.cli batch -config config.yaml`

4.2 Kết quả tổng hợp

4.3 Phân tích chi tiết và thảo luận

4.4 So sánh với baseline và kết luận thực nghiệm

5 Hạn chế và hướng phát triển

5.1 Hạn chế

- **Mô hình hoá đơn giản:** thời gian phục vụ giả định ≈ 0 ; không xét rendezvous / launch-recover truck-drone; drone không bị ràng buộc bán kính bay nếu không cấu hình `radius`; chưa áp dụng giới hạn thời gian bay riêng (chỉ có L_w chung cho tour).
- **Dữ liệu và phạm vi:** chạy trên một instance cố định (ví dụ 50.40.1) từ `data/raw`; chưa có quy trình train/val/test hay đánh giá đa-instance mặc định; cờ `dronable` chỉ mới được áp dụng gần đây (các log cũ có thể chưa phản ánh).
- **Thuật toán:** fitness phải mô phỏng đầy đủ nên tốn thời gian, chưa có song song hoá; GP không có validation nội bộ dễ overfit vào instance/seed; nhạy cảm tham số `pop_size`, `rc`, `rm`; chưa khai thác đặc trưng năng lượng/pin hay rủi ro thời tiết.
- **Tài nguyên/thiết bị:** hàng đợi *pending* và chèn tốt nhất cho từng xe có độ phức tạp cao trên instance lớn, dễ quá tải CPU/RAM; chưa có tối ưu tính toán hay ràng buộc cứng để giảm tải.

5.2 Hướng phát triển

- **Mở rộng ràng buộc thực tế:** thêm bán kính/tầm bay và giới hạn thời gian bay riêng cho drone; mô hình pin/charging, thời tiết, huỷ đơn; xem xét nhiều depot, rendezvous truck-drone.
- **Cải tiến tối ưu hoá:** song song hoá tính fitness, cache mô phỏng; hybrid GP + local search/ALNS; ensemble nhiều luật; tự điều chỉnh `rc/rm/pop_size` động theo thể hệ.
- **Đánh giá và khái quát:** chạy đa-instance/batch, thiết lập validation hoặc cross-instance để chọn mô hình; ablation đặc trưng; so sánh với benchmark công khai, kiểm định thống kê và phân tích nguyên nhân drop/robustness theo seed.

5.3 Kết luận

References

- [1] C. C. Murray and A. G. Chu, “The flying sidekick traveling salesman problem: Optimization of drone-assisted parcel delivery,” *Transportation Research Part C: Emerging Technologies*, 2015.
- [2] G. Macrina et al., “Drone-aided routing: A literature review,” *Transportation Research Part C: Emerging Technologies*, 2020.
- [3] Y. J. Liang et al., “A survey of truck–drone routing problem: Literature review and research prospects,” *Journal of Operations Research and Control*, 2022.
- [4] H. N. Psaraftis and M. Wen, “Dynamic vehicle routing problems: Three decades and counting,” *Networks*, 2015.
- [5] O. Bräysy and M. Gendreau, “Vehicle routing problem with time windows, part ii: Metaheuristics,” *Transportation Science*, 2005.
- [6] D. Jakobović, “Evolving dispatching rules for dynamic vehicle routing using genetic programming,” *Algorithms*, 2023.
- [7] J. Jacobsen-Grocott, Y. Mei, G. Chen, and M. Zhang, “Evolving heuristics for dynamic vehicle routing with time windows using genetic programming,” in *IEEE Congress on Evolutionary Computation (CEC)*, 2017.
- [8] M. A. Nguyen et al., “The min-cost parallel drone scheduling vehicle routing problem,” *European Journal of Operational Research*, 2022.
- [9] J. Xing et al., “Reliable truck-drone routing with dynamic synchronization,” *Transportation Research Part C: Emerging Technologies*, 2024.
- [10] J. Schmidt et al., “The vehicle routing problem with drones and time windows,” *Tech. Rep.*, 2024.
- [11] J. Han et al., “Vehicle routing problem with drones considering time windows and dynamic demand,” *Applied Sciences*, 2023.
- [12] J. MacLachlan et al., “Genetic programming hyper-heuristics with vehicle collaboration for uncertain capacitated arc routing problems,” *Evolutionary Computation*, 2020.